CL2001 — Data Structures

Spring 2026

Intelligent Forest Advisory & Multi-Structure Decision System (IFAMDS)

PROJECT REPORT

Submitted By

Group of Two | Section: _____

Submission Date: May 9, 2026

Department of Computer Science

1. System Overview

IFAMDS (Intelligent Forest Advisory and Multi-Structure Decision System) is a fully console-based C++ simulation developed for the CL2001 Data Structures course. The project models a real-world forest management environment where sensor data flows through multiple processing layers and decisions emerge from the combined interaction of all major data structures rather than any single isolated component.

The system was architected across eight departments, each responsible for a distinct computational concern:

- Department 1 — Environmental Data Acquisition (Arrays A1–A4)
- Department 2 — Event Memory and Temporal Reconstruction (Linked Lists L1–L10)
- Department 3 — Execution Control and Computational Reasoning (Stack-based ECL)
- Department 4 — Task Scheduling and Priority Allocation (Queues Q1–Q4 + DSCH)
- Department 5 — Hierarchical Decision Intelligence (Trees T1–T12)
- Department 6 — Spatial Connectivity and Routing (Graphs G1–G2)
- Department 7 — Indexing and Retrieval Acceleration (Hash Tables H1–H3)
- Department 8 — System Monitoring and Adaptive Optimization (AMON)

In its final form the system spans roughly 4,700 lines of C++ with zero STL container usage. Every data structure — stacks, queues, priority queues, adjacency lists, matrices — was built from scratch using fixed-size arrays and custom logic. The code compiles cleanly with g++ -std=c++17 and passes all six scenario simulations without any runtime errors.

2. Data Structure Usage Summary

2.1 Arrays (A1 – A4)

Four array structures form the input backbone of the system. A1 is a static baseline array holding normal forest conditions (temperature 25 °C, humidity 60 %, smoke level 0) for all ten zones. A2 is the live dynamic sensor array that gets updated during runtime. A3 is a two-dimensional static grid matrix (4 × 4) that stores spatial temperature readings per cell. A4 is a dynamic terrain risk matrix whose values change as conditions worsen.

Arrays gave us O(1) random access by zone index, which was critical for the consistency checks in the ECL module. The 2D arrays in A3 and A4 enabled the spatial interpolation function — when a sensor fails, the missing value is estimated as the arithmetic mean of its four grid neighbours.

2.2 Linked Lists (L1 – L10)

Ten linked list instances cover three structural variants. L1 through L3 are singly linked lists used for raw event streams, verified event streams, and anomaly logging respectively. L4 through L6 are doubly linked lists that support both forward propagation (correcting downstream readings when a past value is fixed) and backward correction (tracing back to the point of inconsistency). L7 through L10 are circular linked lists that run continuous monitoring loops — local, system-wide, emergency, and stability.

The choice of linked lists over arrays for event storage was deliberate. Sensor readings arrive in bursts and need to be appended efficiently. Singly linked insert-back runs in O(n) but maintains chronological order without pre-allocating space. The doubly linked lists were the most algorithmically interesting component because they required traversal in both directions, and designing the backward

correction function to skip the "source" node before applying changes backward took several iterations to get right.

2.3 Stacks (ECL — Dept. 3)

The Execution Control Layer uses two stacks. The checkpoint stack stores full SystemSnapshot objects — a complete deep copy of the A2 sensor array — every time the system is about to enter a potentially dangerous state. If a consistency check fails, rollback() pops the most recent snapshot and restores A2, ensuring the system never operates on corrupt data. The step log stack records every processing action as an ExecutionStep record so there is always a complete audit trail that can be inspected from newest to oldest.

Using a stack for checkpoints was a natural fit because rollback always targets the most recently captured safe state. The LIFO property means the newest snapshot is always on top and the pop operation is O(1).

2.4 Queues (Q1 – Q4 + Dept. 4)

Four queues handle task scheduling. Q1 (Routine Monitoring) and Q2 (Continuous Surveillance) are standard FIFO queues. Q4 (Multi-Factor Decision) is also FIFO, holding tasks that require combining multiple sensor inputs before a decision can be made. Q3 (Emergency Response) is a priority queue — internally implemented as a binary min-heap — where the task with the lowest priority number (highest urgency) is always dequeued first.

The Dynamic Scheduler (DSCH) sits on top of these queues and performs two operations: adjustPriorities (scanning Q1 for tasks in fire zones and escalating them to Q3) and rebalanceQueues (redistributing work from Q1 to Q2 when one queue becomes overloaded). Both operations drain and refill the queues using local arrays rather than STL containers.

2.5 Trees (T1 – T12)

Twelve decision trees cover the complete range of forest management concerns: structural zone decomposition (T1–T3), resource availability (T4–T6), incident classification (T7–T9), and hierarchical decision execution (T10–T12). Each tree is built with a generic TreeNode that holds a name, a risk score, and up to fifteen child pointers.

Tree traversal is pre-order recursive. The evaluateDecision method computes a weighted risk score (Score = w1 × fire + w2 × smoke + w3 × temperature) and compares it against a configurable threshold. T10 handles local zone-level decisions, T11 escalates to neighbouring zones if fire spread rate exceeds 0.5, and T12 issues a global alert when the aggregate risk across all zones crosses the system-wide threshold.

2.6 Graphs (G1 – G2)

G1 is an adjacency list graph implemented as a 2D array of Edge structs (destination + cost) with a separate count per vertex. G2 is a dense adjacency matrix stored as a plain double[10][10] array. G1 is used where connectivity is sparse (mountain paths); G2 is used for full-grid dense checks.

Both graphs support BFS and G1 additionally supports DFS. BFS is used to predict level-by-level fire spread from any source zone. DFS explores a single deep path through the forest to trace rescue

routes or analyze fire corridors. The fire-aware cost update multiplies existing edge weights by (1 + fireLevel) making burning corridors prohibitively expensive for routing algorithms.

2.7 Hash Tables (H1 – H3)

H1 uses open addressing with linear probing. The hash function is ZoneID mod 11 (table size is prime for better distribution). H2 uses separate chaining — each bucket is the head of a singly linked chain of ChainNodes, resolving collisions without probing. H3 is a simple direct-mapped cache that stores the most recently accessed zone data at slot (ZoneID mod 11) for O(1) repeated lookup.

The three-tier approach mirrors a real database design: primary index for main data, collision table for conflict resolution, and cache for hot-path acceleration. During Scenario 4 (System Overload), the H3 cache warm-up step was critical because it eliminated repeated H1 linear probes during peak load.

3. Scenario Descriptions

3.1 Scenario 1 — Cascading Fire and Resource Conflict

A fire starts in Zone 3 and spreads toward Zones 4 and 6. The system captures a pre-fire ECL checkpoint, updates A2 with escalating sensor readings, validates each reading against the A1 baseline, and stores all events in L1. Readings that violate the consistency threshold trigger a rollback. After restoration, DSCH scans Q1 and escalates fire-zone tasks to Q3. T7 calculates a weighted fire score for each zone and T10 determines the response level. G1 BFS from Zone 3 predicts the spread path level by level. T11 triggers regional escalation when the spread rate exceeds 0.5. All confirmed data is written to H1 and synchronized via L4/L6.

3.2 Scenario 2 — Sensor Failure and System Reconstruction

Several sensors in Zone 2 begin returning corrupt or physically impossible values. The verified stream (L2) rejects all bad readings, leaving the stream empty. The ECL step log is scanned backward to locate the last valid Zone 2 entry. A rollback restores the pre-failure state. Missing values are then reconstructed using a combined estimate from A3 spatial interpolation and the last known good value from the step log. If the reconstructed value still fails consistency checking, a second rollback applies the strict baseline. L5 backward correction fixes the historical event chain and H1 is updated with the final restored value.

3.3 Scenario 3 — Multi-Factor Anomaly Escalation

Three different types of anomalies appear simultaneously: abnormal wildlife movement in Zone 8 (T8), elevated fire risk in Zones 1 and 5 (T7), and unknown human presence in Zone 7 (T9). Each anomaly type is classified independently. A combined multi-factor danger score is then computed using weighted contributions from all three signal types. Zones scoring above 0.6 are escalated to L3 and Q3; others continue normal observation via Q1 and L7. BFS from affected zones checks neighbouring areas for early warning. T12 evaluates whether the combined situation warrants a global alert.

3.4 Scenario 4 — System Overload and Load Redistribution

A burst of sensor updates floods all four queues simultaneously. The ECL captures a safe state checkpoint. DSCH immediately scans Q1, separates critical fire-zone tasks, and moves them to Q3 while keeping routine tasks waiting. DSCH then rebalances Q1 and Q2 if their sizes are too far apart. H3 is pre-warmed with the most frequently queried zones to avoid repeated hash table lookups during peak load. Paused Q1 and Q2 tasks are then drained in controlled batches of three. AMON detects the worst-performing module, redistributes load by 15 %, tunes thresholds based on current load levels, and confirms the system has returned to normal via L10.

3.5 Scenario 5 — Global Multi-Zone Emergency Synchronization

A large-scale emergency pushes conflicting readings across all ten zones simultaneously. ECL runs a full consistency scan and classifies zones into conflicting and clean sets. Conflicting zones are isolated in L3. L5 backward correction and an ECL rollback restore a globally coherent state. Correction values are computed as the average of the A1 baseline and A3 spatial interpolation, then propagated forward via L4 and synchronized across all modules via L6. T11 and T12 resolve regional and global decision conflicts. G2 matrix BFS runs from each confirmed hot zone to generate synchronized response paths. Q3 receives a bulk dispatch of emergency tasks. L8 system-wide circular loop and the AMON full health report confirm alignment.

3.6 Scenario 6 — Execution Control and State Recovery

This scenario exercises the three control departments in isolation. Corrupt sensor values are deliberately injected into specific zones. ECL detects them zone by zone, pauses execution on the first violation, rolls back to the checkpoint, and replaces remaining bad zones with interpolated values from A3. DSCH re-runs priority adjustment after recovery. A load-balance report is generated, followed by queue rebalancing. AMON then runs the full bottleneck detection, threshold tuning, load redistribution, and health report pipeline. The ECL step log and checkpoint stack are displayed in full.

4. Development Log (1 – 8 May 2026)

The following table records the daily work done on this project between May 1 and May 8. Times are approximate and represent actual productive coding and debugging sessions, not total time at the computer.

Date	Hours	Work Done
Thu 1 May	3h 40m	Set up project structure. Implemented A1, A2 (static and dynamic sensor arrays) with initialize() and updateZone() methods. Wrote A3 static grid with interpolate() and detectBoundaries(). Hit an issue where interpolate() was reading out-of-bound indices — fixed by adding r > 0, r < GRID_ROWS guards. Tested A4 terrain matrix update.
Fri 2 May	4h 15m	Built all ten linked list classes (SinglyLinkedList, DoublyLinkedList, CircularLinkedList). Most time went into the doubly linked backward correction — first version was overwriting the source node itself which broke data. Fixed by setting applying flag only after the source node was matched.

Date	Hours	Work Done
		Circular list destructor also had a dangling pointer bug; fixed by tracking head separately before the loop.
Sat 3 May	2h 50m	Implemented all four queues. ForestQueue was straightforward. EmergencyQueue using std::priority_queue was quick but later had to be replaced entirely. Built the Dynamic Scheduler (DSCH) adjustPriorities and rebalanceQueues methods. Spent about 45 minutes debugging rebalanceQueues because it was silently losing tasks when both queues were empty — added an early return guard.
Sun 4 May	4h 30m	Built all 12 decision trees with addChild and evaluateDecision. Pre-order traversal worked first try. Then built both graph classes — AdjListGraph and AdjMatrixGraph. BFS worked immediately. DFS had a subtle bug where the recursive visited array was not passed by reference, so nodes were revisited. Fixed by passing bool visited[] as a pointer parameter. Fire-aware cost update tested on Zone 3.
Mon 5 May	3h 20m	Implemented H1 (open addressing with linear probing), H2 (separate chaining), and H3 (direct-mapped cache). Hash collision test: Zone 3 and Zone 14 both map to index 3 — chaining handled it cleanly. Built the ECL (Execution Control Layer) with checkpoint stack, step log, rollback, and alternative path. Tested rollback with a deliberately corrupt A2 value. Confirmed zones restored correctly after pop.
Tue 6 May	2h 10m	Built all six scenarios step by step. Scenarios 1 and 2 worked mostly on the first pass. Scenario 3 had a range-for over a local struct array that was giving warnings about structured binding on non-trivial types — refactored to index loop. Scenario 5 had the most logic to trace. The conflictZones / cleanZones separation required careful test to confirm that clean zones were being rebuilt correctly.
Wed 7 May	4h 45m	Replaced all STL containers with custom implementations as per requirement. Replaced std::queue with circular array MyQueue, std::stack with array-based MyStack, std::priority_queue with a binary min-heap, std::vector<pair> with Edge struct array, std::vector<string> with fixed array + size counter. The priority queue replacement was the hardest part — heapifyUp and heapifyDown logic had an off-by-one error where the root node was not being sifted correctly when two tasks had equal priority. Fixed by using strict less-than comparison.
Thu 8 May	3h 05m	Final integration, cleanup, and report writing. Ran all six scenarios in sequence. Fixed a minor issue in the ECL displayStepLog where the at() index was going negative when the step log was empty. Added empty() guard. Checked that all time complexity comments were present on major operations. Wrote function-level comments for every method. Verified the project compiles clean with g++ -std=c++17 -Wall -Wextra with zero errors and zero warnings.

Total time invested: **28 hours 55 minutes** across 8 days.

5. Challenges Faced and How We Solved Them

5.1 Doubly Linked List Backward Correction

The backward correction function (L5) was supposed to walk backward from a given node and overwrite all previous entries for the same zone with a corrected value. Our first implementation set applying = true at the very start of the loop and immediately overwrote the source node itself before moving backward. The result was that the node whose correction triggered the fix was also getting overwritten — defeating the purpose.

The fix was to change the logic so that applying only becomes true after the source node is found, meaning subsequent (older) nodes are corrected while the source retains its new value. This also meant we had to traverse from the tail pointer, not the head, because "backward" in time means toward the beginning of the list.

5.2 DFS Visited Array Scope

During graph DFS, we declared the visited array inside the public runDFS function and passed it to the recursive dfsHelper. But because we originally passed it by value (vector<bool> visited), each recursive call got its own copy of the visited state and the algorithm revisited every node, producing infinite recursion on cyclic connections.

After switching to bool visited[] (a plain array passed as a pointer), the same array was shared across all recursive calls and DFS behaved correctly. This was one of those bugs that is immediately obvious in hindsight but cost a good thirty minutes of tracing output before we saw the pattern.

5.3 Priority Queue Min-Heap Replacement

Replacing std::priority_queue with a custom binary min-heap looked straightforward until we tested with tasks of equal priority. Our heapifyUp was comparing with <= instead of <, which caused swaps between two nodes of equal priority to loop indefinitely in certain configurations. The fix was to use strict < so that equal-priority nodes are never swapped during the sift-up phase, making the heap stable for equal keys.

We also discovered that the display method we wrote for the emergency queue was sorting a temporary copy of the heap array using a simple selection sort. This is O(n^2) but since it only runs for display output and the heap is bounded at 150 tasks, the performance cost was acceptable and the code remained simple.

5.4 ECL Stack Display Without STL Copy

The original displayStepLog and displayCheckpoints methods made a copy of the std::stack and popped from it for iteration. After replacing std::stack with our custom MyStack, we no longer had a stack copy constructor that mirrored std::stack's semantics. Rather than implement one, we exposed an at(i) index accessor on MyStack that reads from the underlying array directly. This let us iterate from topIdx down to 0, which gives newest-first order without needing a copy at all.

5.5 Scenario 5 Global State Inconsistency Logic

Scenario 5 is the most complex: ten zones receive conflicting async updates simultaneously and the system has to identify which zones are clean and which are in conflict, then reconstruct a globally

coherent state. The tricky part was that after rollback, the conflicting zones needed to be corrected using a blend of A1 baseline and A3 spatial interpolation, but the interpolation function takes a row and column, not a flat zone index.

We mapped zone index to grid coordinates using row = zoneID / GRID_COLS and col = zoneID % GRID_COLS. This gave us the correct neighbour-average interpolation for each zone. We tested the edge case where a zone in the first row has no top neighbour — the interpolate() function already handled this by only counting valid neighbours, so the fix was already built in.

5.6 MyQueue Wrap-Around in Circular Buffer

Our circular array MyQueue uses frontIdx, backIdx, and sz to manage the ring. During the DSCH adjustPriorities function, we drain Q1 entirely (popping all tasks) and then push them back in two batches (retained and upgraded). After the first full drain, frontIdx and backIdx both sat at the same position. When we refilled, tasks were written correctly but the first few reads from Q2 after the rebalance were returning stale values from a previous session.

The root cause was that we were not resetting frontIdx and backIdx when sz dropped to zero. We added an explicit reset (frontIdx = backIdx = 0) inside the pop() function whenever sz reaches zero after a dequeue. This ensured the buffer always started clean after being completely drained.

6. Key Design and Logic Decisions

6.1 No STL Containers — Everything from Scratch

The project requirement was to demonstrate correct usage of data structures. Using STL would have meant the data structures were already implemented for us. We therefore replaced all STL containers with hand-written equivalents using fixed-size arrays allocated on the stack. This had a secondary benefit: we could add methods like at(i) for direct index access, which simplified the ECL display logic considerably.

6.2 Template-Based MyStack and MyQueue

Rather than writing a separate stack for SystemSnapshot and another for ExecutionStep, we used a single C++ template class MyStack<T, MAXSZ> that works for any type with a fixed compile-time capacity. The same pattern was used for MyQueue<T, MAXSZ>. Templates keep the code DRY while the fixed MAXSZ parameter eliminates heap allocation and makes capacity violations visible at runtime through printed overflow messages.

6.3 Weighted Decision Score as the Central Logic Primitive

All twelve trees ultimately reduce their decisions to the same formula: Score = w1 × input1 + w2 × input2 + w3 × input3. Threshold checking then maps the score to a binary emergency/normal outcome. This uniformity meant we could implement one evaluateDecision() method on ForestDecisionTree and reuse it for fire classification, wildlife detection, human intrusion, and global emergency — all with different weight profiles but identical logic flow.

6.4 ECL as the System's Nervous System

Most systems would handle sensor validation inline. We deliberately extracted it into a separate class (ExecutionControlLayer) that wraps every major state transition. This means every zone update produces an ExecutionStep record on the step log, every dangerous update is guarded by a checkpoint, and every failure triggers a well-defined rollback or alternative-path response. The architecture made Scenario 6 (dedicated state recovery testing) trivial to implement because all the machinery was already in place.

6.5 Three-Level Hash Architecture

H1 (open addressing) handles the normal insert/search path. H2 (separate chaining) is specifically populated with colliding keys as a demonstration of collision resolution. H3 (cache) sits above both and is explicitly warmed at the start of high-load scenarios. This three-level design mirrors an industry database architecture and gave us natural opportunities to demonstrate all three hashing strategies in a single coherent system.

7. Sample Console Output Highlights

The following excerpts illustrate representative output from the system. Screenshots of full menu execution are included in the submission zip file.

7.1 BFS Fire Spread (Scenario 1, Step 8)

```
[G1-BFS] Fire spread from Zone3:
   Level 0:  Zone3
   Level 1:  Zone1  Zone6
   Level 2:  Zone0  Zone4  Zone8
   Level 3:  Zone2  Zone5  Zone7  Zone9
```

7.2 ECL Rollback and Alternative Path (Scenario 2, Steps 5–6)

```
[ECL] Rolling back to: "SCENARIO2_PRE_FAIL"  (tick=18)
[ECL] State restored. Execution resumed.
[ECL] Alternative Path: Zone2 replacing faulty=55.0
                        with interpolated=26.5
```

7.3 Emergency Queue Dequeue Priority Order (Scenario 5, Step 9)

```
[Q3-EmergencyResponse] EMERGENCY dequeued: SYNC-Z0  Zone=0  Priority=1
[Q3-EmergencyResponse] EMERGENCY dequeued: SYNC-Z2  Zone=2  Priority=1
[Q3-EmergencyResponse] EMERGENCY dequeued: SYNC-Z4  Zone=4  Priority=1
Total synchronized tasks dispatched: 6
```

8. Time Complexity Reference

All major operations in the system carry explicit time complexity comments in the source code. The table below summarises the most important ones.

Data Structure	Operation	Time Complexity
Array (A1–A4)	Single zone access / update	O(1)
Array (A1–A4)	Full scan / anomaly detection	O(n × m)
Array (A3)	Spatial interpolation	O(1)
Singly Linked List	Insert front	O(1)
Singly Linked List	Insert back / search	O(n)
Doubly Linked List	Insert back (at tail)	O(1)
Doubly Linked List	Forward / backward correction	O(n)
Circular List	Insert / one monitoring cycle	O(1) / O(n)
MyStack	Push / pop / peek	O(1)
MyQueue (FIFO)	Enqueue / dequeue	O(1)
EmergencyQueue (PQ)	Enqueue / dequeue (heap)	O(log n)
Tree (T1–T12)	Add child	O(1)
Tree (T1–T12)	Pre-order traversal	O(n)
AdjListGraph (G1)	BFS / DFS traversal	O(V + E)
AdjMatrixGraph (G2)	BFS traversal	$O(V^2)$
AdjMatrixGraph (G2)	Edge lookup	O(1)
Hash Table H1 (OA)	Insert / search (average)	O(1)
Hash Table H2 (SC)	Insert / search (average)	O(1)
Hash Table H3	Cache read / write	O(1)
ECL Checkpoint	Capture (deep copy)	O(n zones)
ECL Rollback	Restore state	O(n zones)

9. CL2001 Requirement Compliance Checklist

Requirement	Status	Where Implemented
C++ console-based simulation	✓ Complete	main() + all menu handlers
Menu-driven hierarchical interface	✓ Complete	Sections 6 menus 1–10 + sub-options
Arrays — static A1, A3	✓ Complete	Section 1: StaticBaseline, StaticGridMatrix
Arrays — dynamic A2, A4	✓ Complete	Section 1: DynamicSensorArray, DynamicTerrainMatrix
Linked Lists — singly (L1–L3)	✓ Complete	Section 2: SinglyLinkedList × 3
Linked Lists — doubly (L4–L6)	✓ Complete	Section 2: DoublyLinkedList × 3
Linked Lists — circular (L7–L10)	✓ Complete	Section 2: CircularLinkedList × 4
Stacks	✓ Complete	Section 7A: ECL checkpointStack + stepLog
Queues FIFO (Q1, Q2, Q4)	✓ Complete	Section 3: ForestQueue × 3
Priority Queue (Q3)	✓ Complete	Section 3: EmergencyQueue (min-heap)
Trees — 12 instances (T1–T12)	✓ Complete	Section 4: ForestDecisionTree × 12
Graphs — adjacency list (G1)	✓ Complete	Section 5: AdjListGraph
Graphs — adjacency matrix (G2)	✓ Complete	Section 5: AdjMatrixGraph
Hash — open addressing (H1)	✓ Complete	Section 6: PrimaryHashTable
Hash — separate chaining (H2)	✓ Complete	Section 6: ChainHashTable
Hash — cache (H3)	✓ Complete	Section 6: FastCache
BFS algorithm with comments	✓ Complete	G1.BFS(), G2.BFS()
DFS algorithm with comments	✓ Complete	G1.runDFS(), dfsHelper()
Hashing algorithm with comments	✓ Complete	hashFunc() in H1, H2, H3
Scheduling with priority	✓ Complete	Q3 min-heap + DSCH
5+ complete scenarios	✓ Complete	Scenarios 1–6 (6 total)
Time complexity on all major ops	✓ Complete	Inline comments throughout code
Function-level comments	✓ Complete	Every method documented
No STL containers	✓ Complete	MyStack, MyQueue, min-heap, Edge array
Modular readable code	✓ Complete	8 departments, clear section headers
Short report (this document)	✓ Complete	You are reading it